Building a 360° Surround View from 4 Fisheye Cameras

A beginner's tour through everything we did — and why — on the way from raw sensor files to a clean top-down view.

1. The setup: 4 cameras, 1 car
2. How a fisheye lens sees the world
3. The core trick: ask backwards, don't warp
4. Blending 4 views into 1 picture
5. Fixing frames that weren't really synchronized
6. Why far-away things looked smeared and blocky
7. Matching brightness across cameras
8. A dead end, investigated properly (feature matching)
9. The two feature-matching techniques, in detail
10. Cleaning up sensor grain
11. Removing the car's own reflection from its view
12. The 'bowl' trick used by real car displays
13. Trying real depth estimation (and why it broke)
14. Fixing double images at the seams

Every page after this one explains ONE of these steps with a simple diagram.

STEP 1

Four Eyes Around a Car

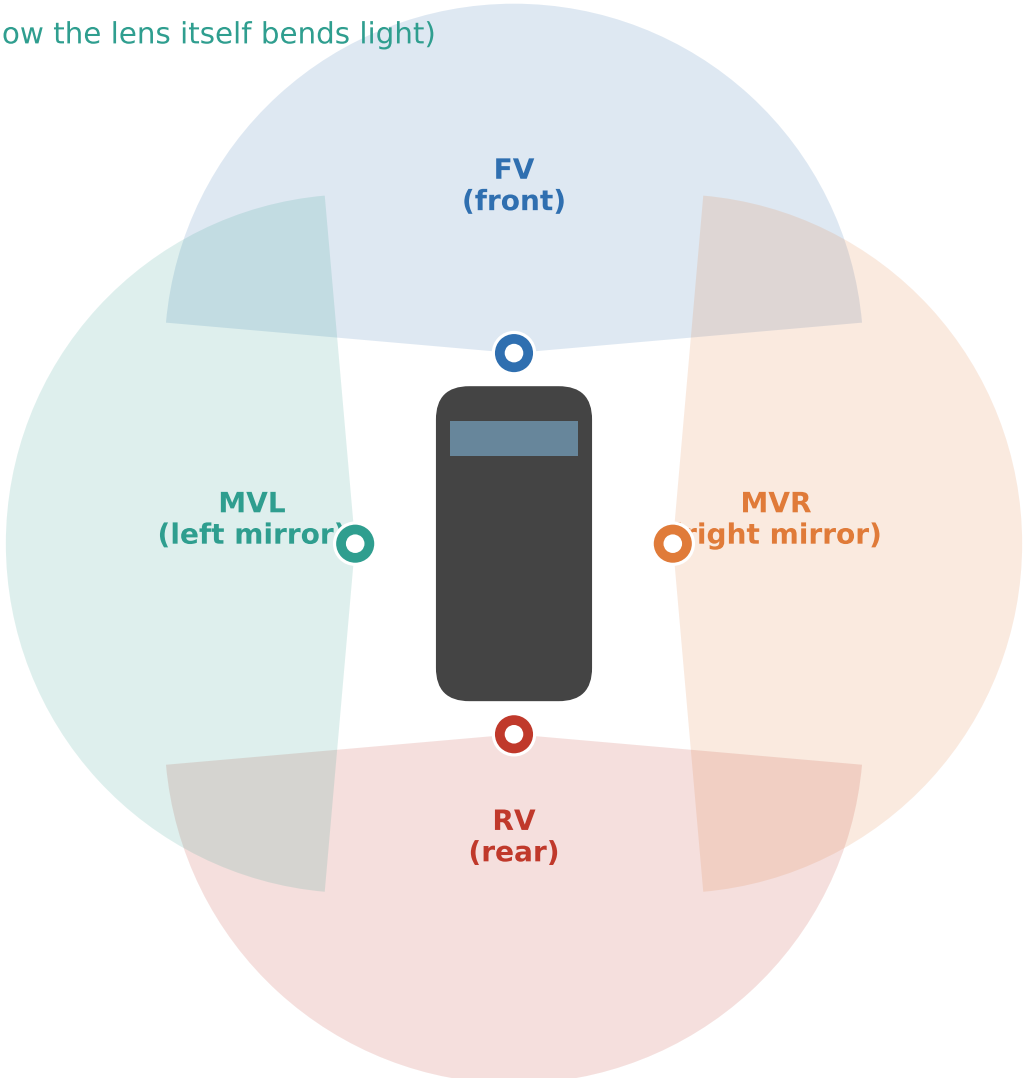
Each camera comes with a file describing exactly where it is and which way it looks

Calibration file =

EXTRINSIC (where it is + which way it points)

+

INTRINSIC (how the lens itself bends light)



The shaded fans show roughly what each camera can see (~180° each) — notice the corners where two cameras' views overlap. That overlap is where most of our later problems (and fixes) happen.

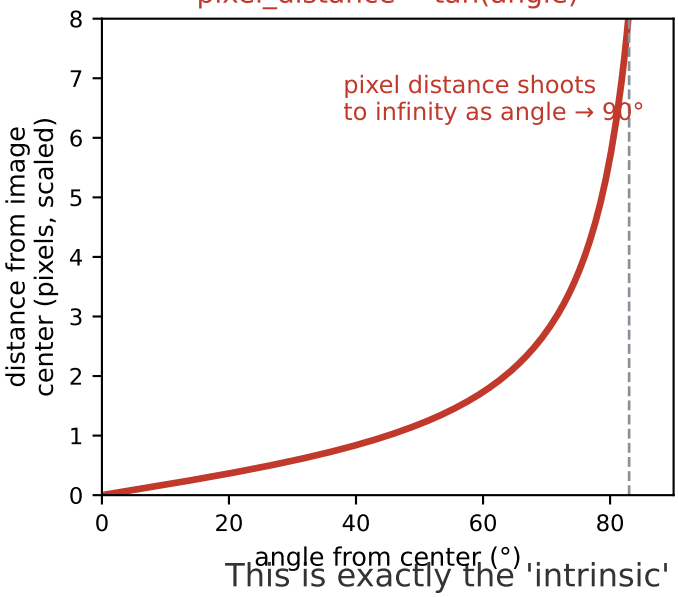
We reused Valeo's own calibration math instead of re-deriving fisheye geometry from scratch.

STEP 2

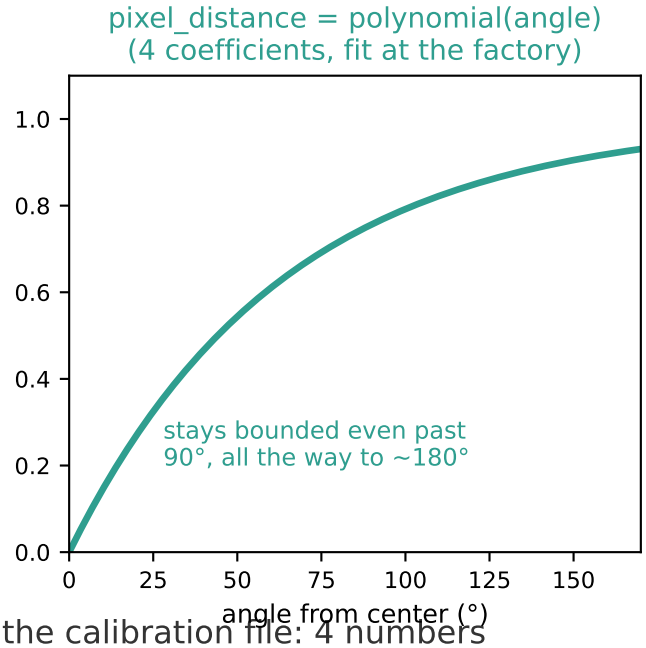
How a Fisheye Lens Sees the World

Why an ordinary camera model can't describe a ~180° lens

Ordinary (pinhole) lens



Fisheye lens



This is exactly the 'intrinsic' part of the calibration file: 4 numbers (k1..k4) that describe this curve for that specific physical lens.

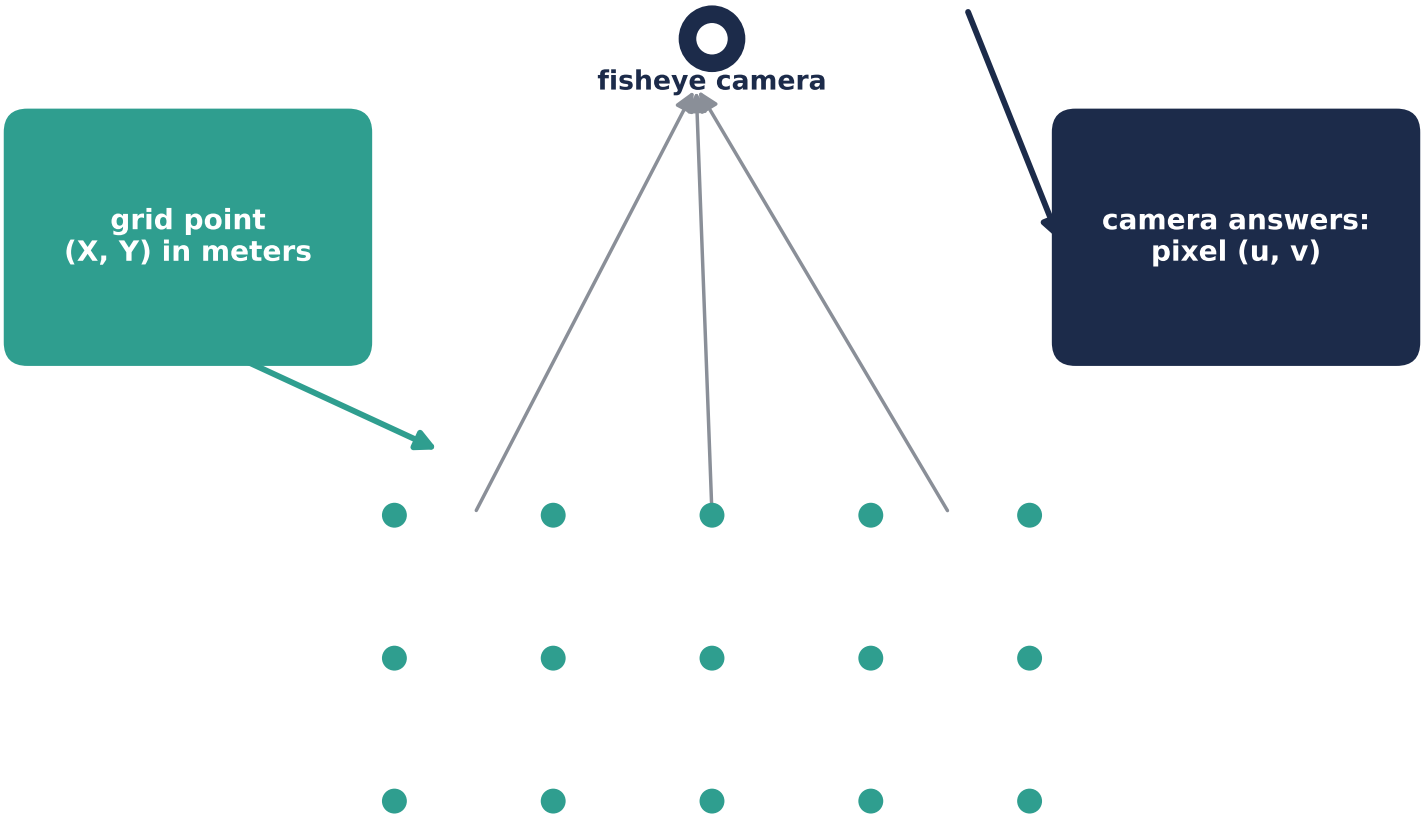
This is why a single flat 'homography' trick (normal panorama software) can't be used here.

STEP 3

The Core Trick: Ask Backwards

Instead of warping the photo, we ask each camera a question for every ground point

"*which of your pixels shows THIS exact ground point?*"



a flat grid of real-world (X, Y) points laid on the ground around the car

WRONG WAY — Old way: warp the WHOLE photo with one flat transform — breaks on curved fisheye lines

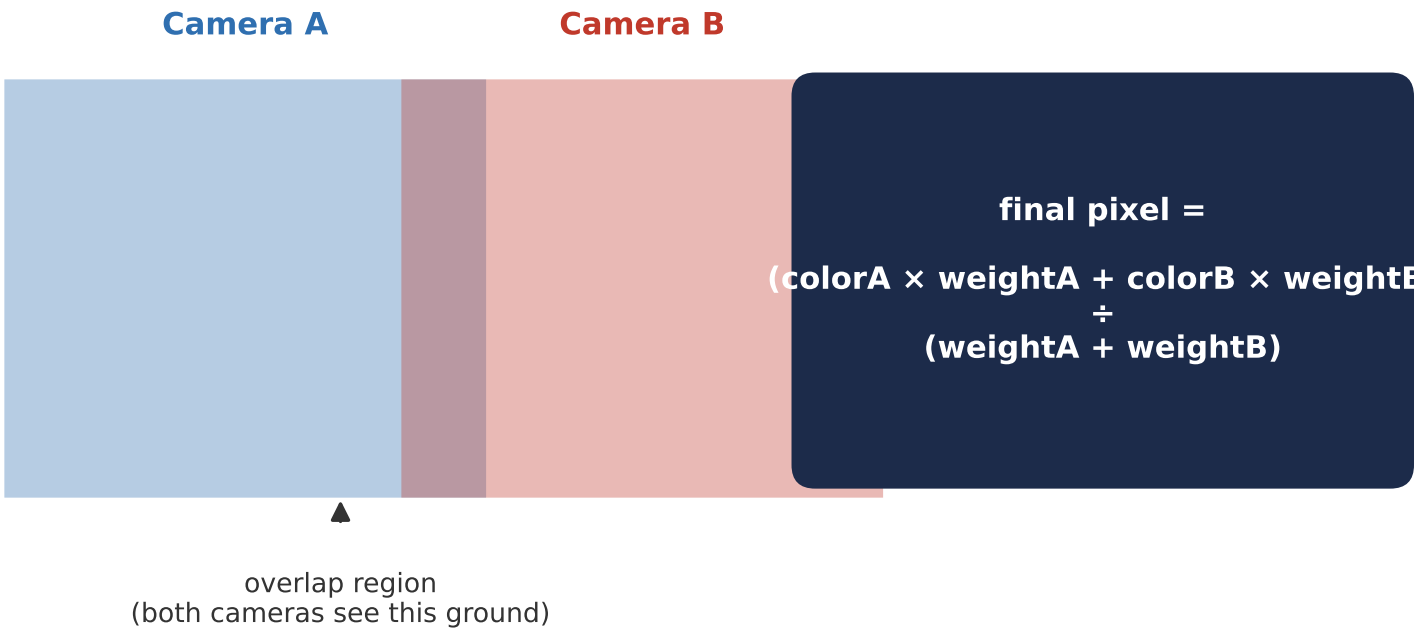
BETTER WAY — Our way: answer this question for every single grid point — works for ANY lens shape

cv2.remap() then just fetches that pixel. Since all 4 cameras use the SAME grid, their answers are automatically aligned.

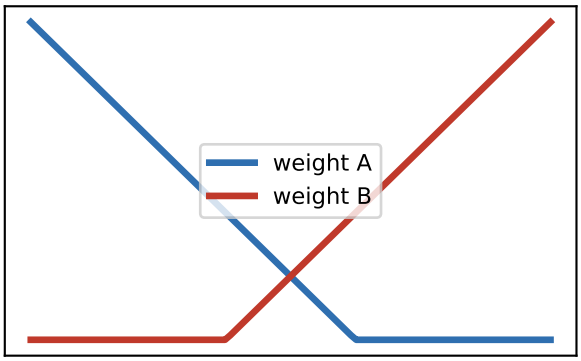
STEP 4

Blending 4 Views Into 1 Picture

Where two cameras' views overlap, fade smoothly between them instead of a hard cut



each camera's confidence, left to right

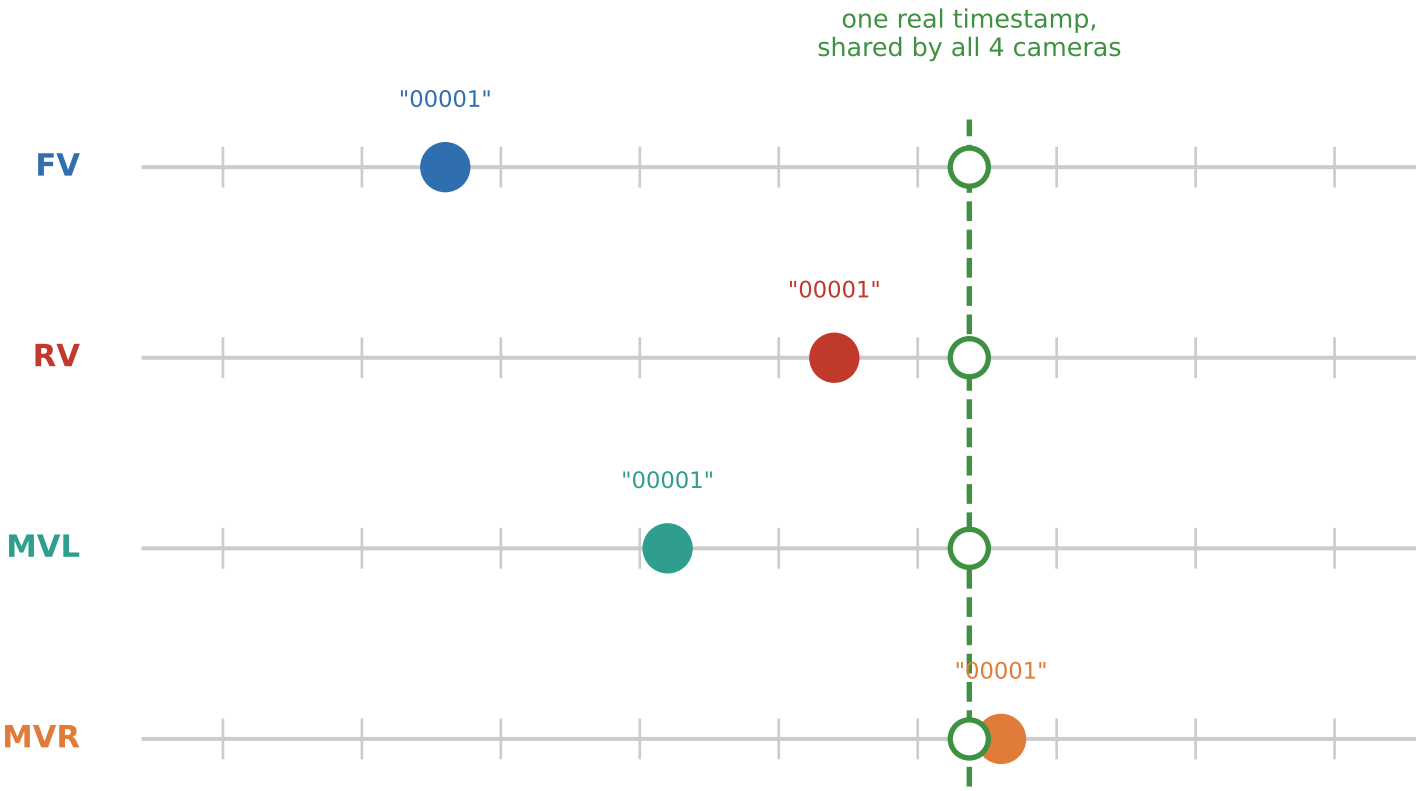


This is basic feathering — every panorama tool does something like this. Getting the WEIGHTS right is where the real work is (see step 14).

STEP 5

Problem: The 4 Cameras Weren't Actually Synce

Same filename number ≠ same moment in time — we had to prove this, not assume it



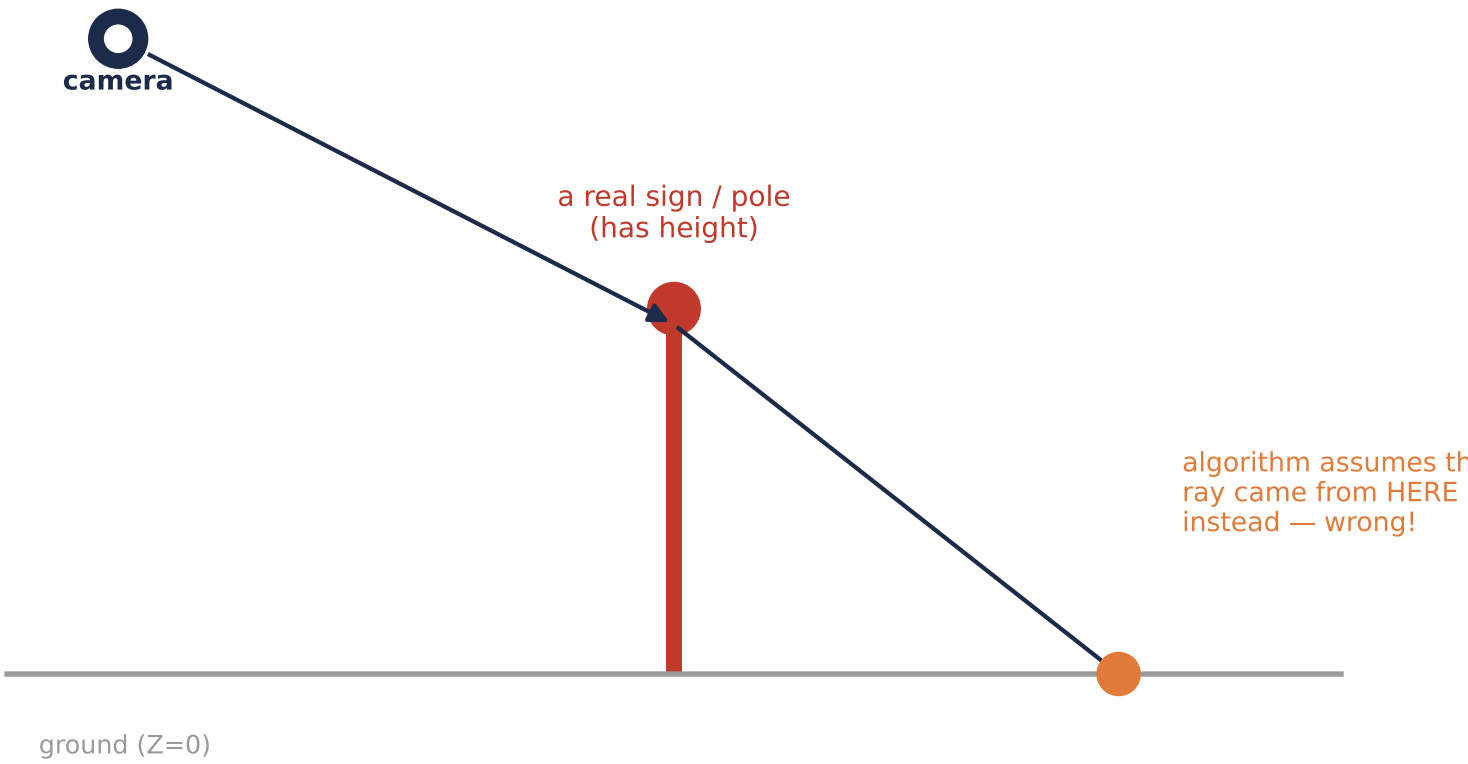
Every camera's OWN frame counter — "00001" happens at a different real time for each one!

Fix: match frames by the hidden 'timestamp' field in vehicle_data/.json, not by filename number.*

STEP 6a

Problem: Tall Things Get Smeared

Our whole trick assumes everything is lying flat on the ground — tall things break that



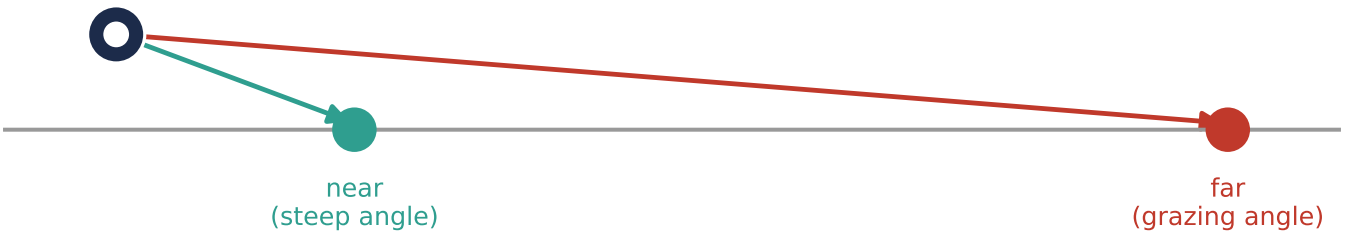
Result: the sign's color gets painted far from where it really is —
the classic radial 'smear' you see on cars, poles, signs, curbs in every render.

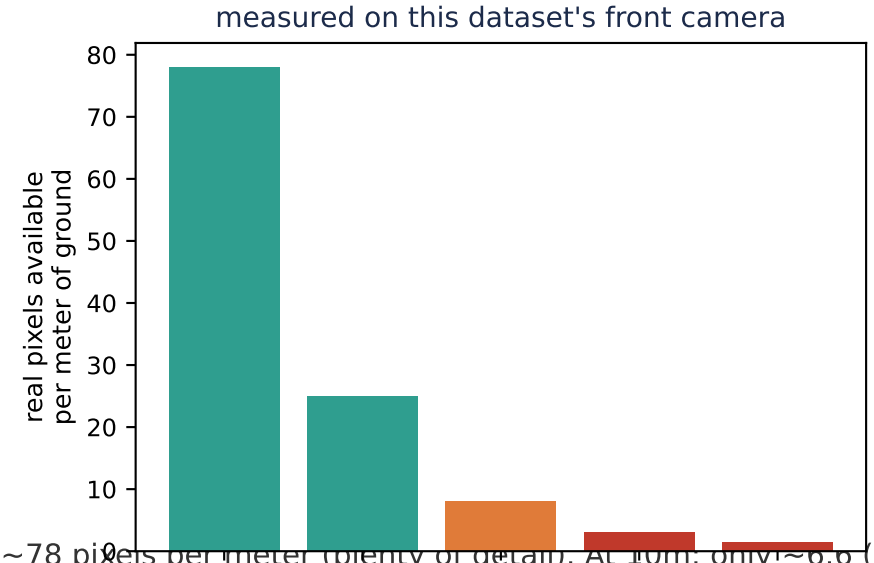
This can't be fixed with better calibration — it's a fundamental limit of the flat-ground assumption itself.

STEP 6b

Problem: Distant Ground Looks Blocky

A second, unrelated cause of 'stretching' — measured directly, not guessed





At 2m: ~78 pixels per meter (plenty of detail). At 10m: only ~6.6 (almost none). We were asking for a sharp 20m-wide picture from a camera that only has that much detail near itself.

Fix: shrink how far out we render (extent_m) to match what the camera can actually resolve.

STEP 7

Problem: Cameras Disagree on Brightness

Each camera auto-exposes independently — like 4 phones each picking their own brightness

Comparing brightness only in the small patch two cameras share didn't work — that patch mixes near and far ground seen at different angles, so different camera-pairs disagreed about which one was "really" too dark.

k used instead: nudge each camera's OVERALL average brightness towa the group average. Cruder, but robust — and it worked.

Same idea as fixing 4 differently-exposed photos before making a collage.

STEP 8

A Dead End, Investigated Properly

The textbook fix for seam ghosting is feature matching — we tried it, and mostly ruled it out

Attempt 1: match features on the flattened top-down image, warp one camera to fit

FAILED — Asphalt has almost no texture to match → warp tore holes in the image

Attempt 2: same idea, but mathematically constrained (rotation + shift only, no tearing)

OK — Safe, but barely helped

The REAL finding: fitted correction was TINY (< 0.3° rotation, < 20cm position)

→ the calibration was already close to correct. The leftover ghosting is mostly Step 6a (height), not a calibration bug at all.

(see the next page for exactly how each attempt worked)

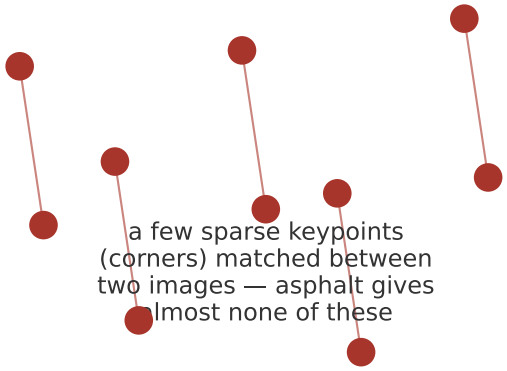
A cheap experiment that tells you "this isn't the problem" is just as valuable as one that fixes something.

STEP 9

The Two Feature-Matching Techniques, In Detail

What "feature matching" actually means here, and why the safer version is so different

Attempt 1: ORB + Homography



a few sparse keypoints (corners) matched between two images — asphalt gives almost none of these

Correspondence: sparse ORB keypoints + BFMatcher (Hamming distance)

Transform fit: full homography (RANSAC) — allows rotate+scale+SKEW

Applied by warping the OUTPUT image → can pull content off-canvas (holes)

Attempt 2: Template Matching

many small patches, each searched for its best match nearby — works even where no single patch stands out

Correspondence: dense normalized cross-correlation (cv2.matchTemplate)

Transform fit: similarity only (RANSAC) — rotate+scale+shift, NO skew

Applied by shifting the INPUT query grid → always samples validly (no holes)

Same underlying idea (find matching ground points, use them to correct calibration) — but the SECOND version is deliberately too constrained to ever produce the tearing failure the first one did.

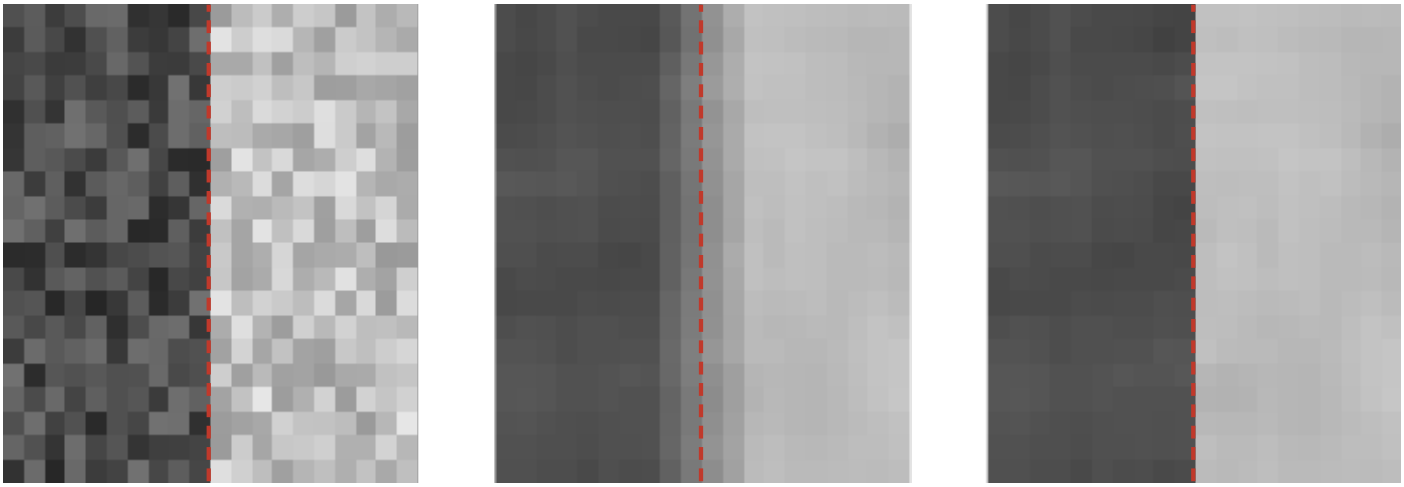
Both are legitimate techniques from classical computer vision — the difference is what each one is allowed to distort.

STEP 10

Cleaning Up Sensor Grain

The grain was real (asphalt texture + sensor noise) — our own pipeline was magnifying it

simulated noisy patch with a real brightness edge down the middle
(dashed red line marks where the true edge is)



before

plain blur
(blurs the edge too!)

bilateral filter
(edge stays sharp)

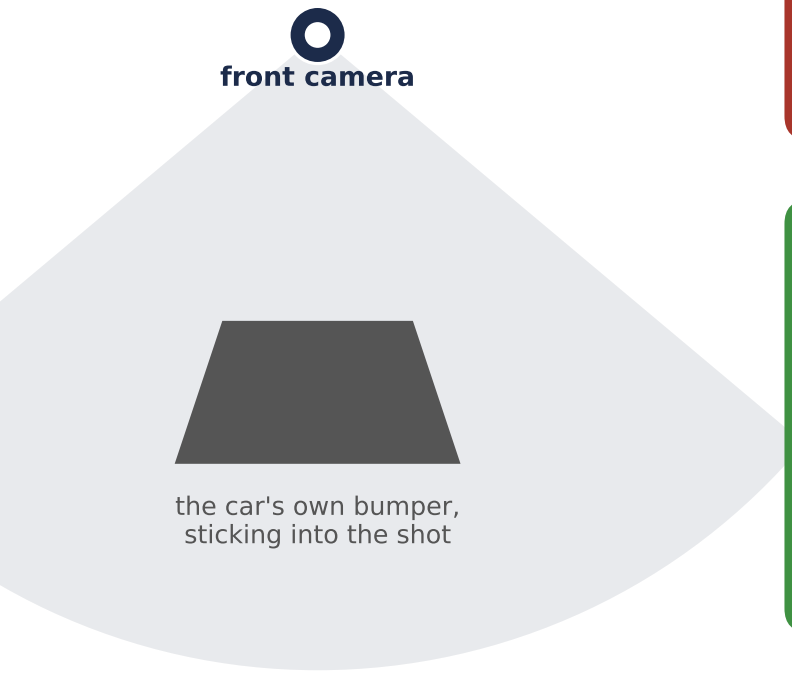
Bilateral filter rule: only average nearby pixels that are BOTH close in position AND similar in color — so flat grain smooths out, but a real edge (very different color) stays sharp.

Applied to the FINAL picture, not the raw camera image — a blur applied before projection barely reached the noisiest (closest-to-car) areas.

STEP 11

The Car Photographing Its Own Reflection

That grey blob in every frame wasn't noise — it was the car seeing its own hood/door



FAILED — Tried: find pixels that are IDENTICAL across many frames → only caught the camera's own dead corners.

Paint/chrome reflect the changing sky, so even the car's own body looks a bit different every frame.

WORKED — AVERAGE many frames together. The changing world blurs into a flat haze, while anything truly fixed in position (even license-plate text!) stays crisp.

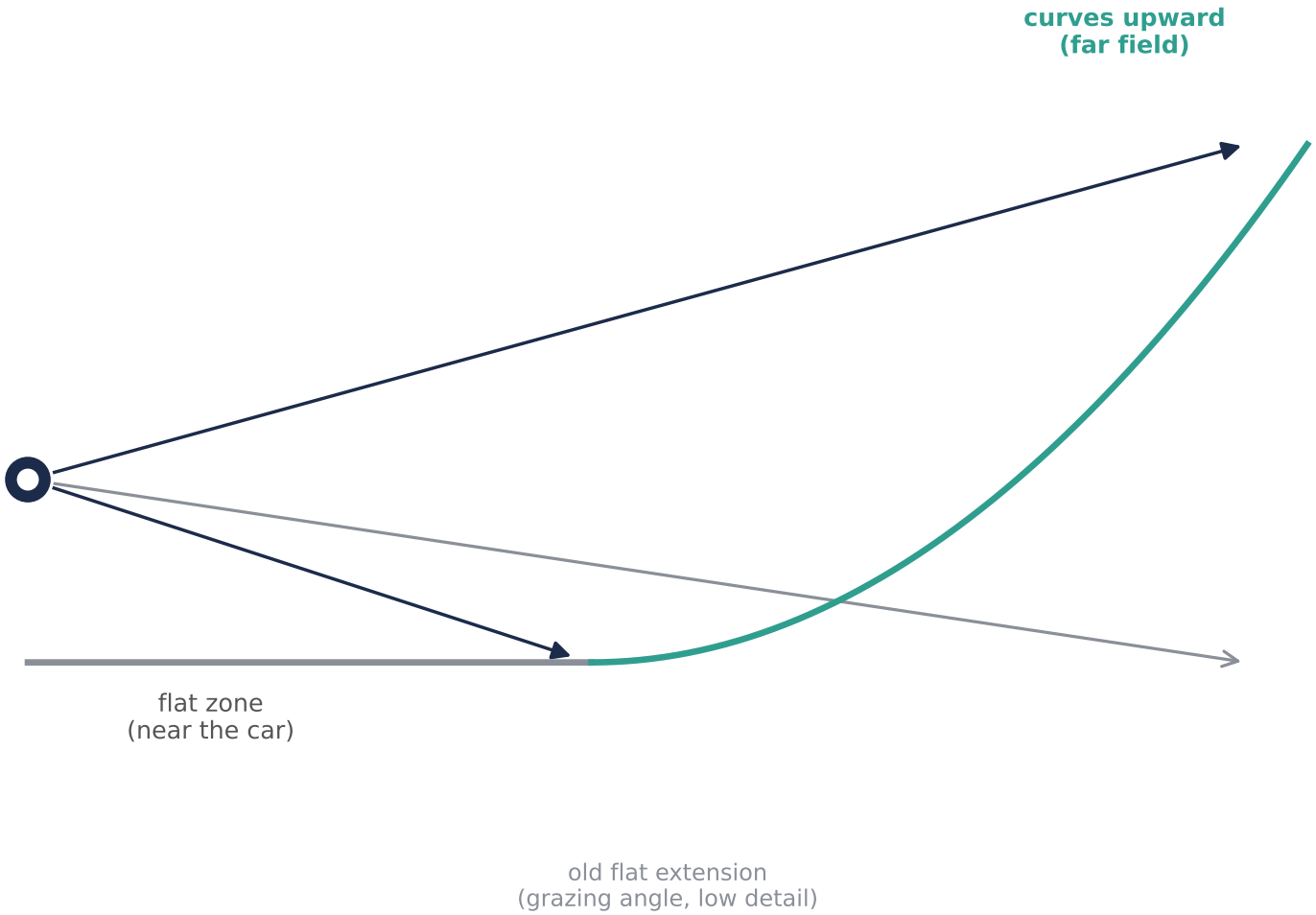
Read that shape off once, hard-code it — exactly how real cars calibrate this.

A one-time mask per camera, since the mount never moves relative to the lens.

STEP 12

Fix: The 'Bowl' Trick Real Cars Use

Instead of a flat ground, assume a shallow bowl — flat near the car, curving up at the edges



Two wins at once: distant blur gets curved out of the main view, AND the ray now hits at a less grazing angle — directly helping the resolution problem from the last page too.

This is literally what production '360 camera' displays in real cars use — not because it's geometrically correct, but because it looks right.

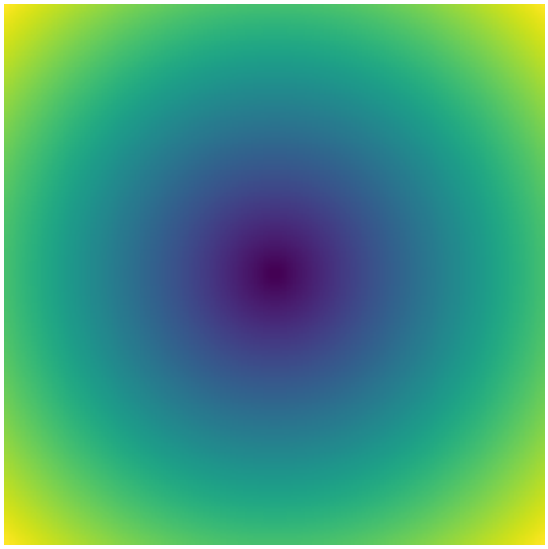
STEP 13

Trying Real Depth Estimation

The 'correct' fix for Step 6a — tried, and failed in a very instructive way

what the depth model learned
from ordinary photos:
"top = far, bottom = close"

what's ACTUALLY true for a
180° fisheye photo:
depth varies in RINGS from the center



Feeding a fisheye image into a model that assumes the left pattern produces a badly wrong depth map — which we then measured directly: it came out as an almost featureless top-to-bottom gradient.

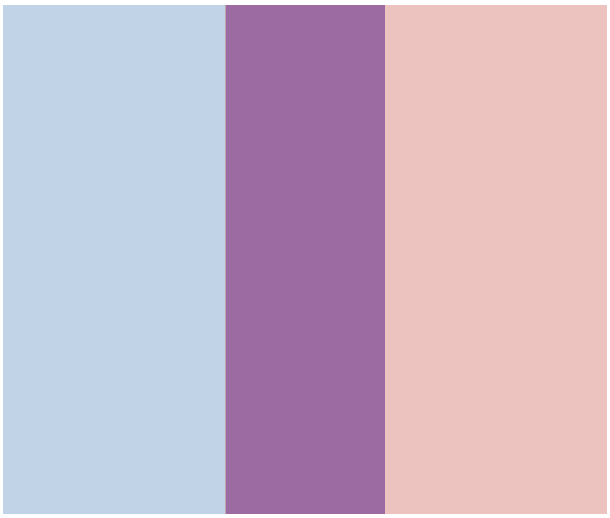
esult when we tried to use it: everything collapsed into ring/donut shap stead of a real scene. Root tool exists (OmniDet, trained on fisheye vide specifically) — just not integrated this session.

STEP 14

Fixing Double Images at the Seams

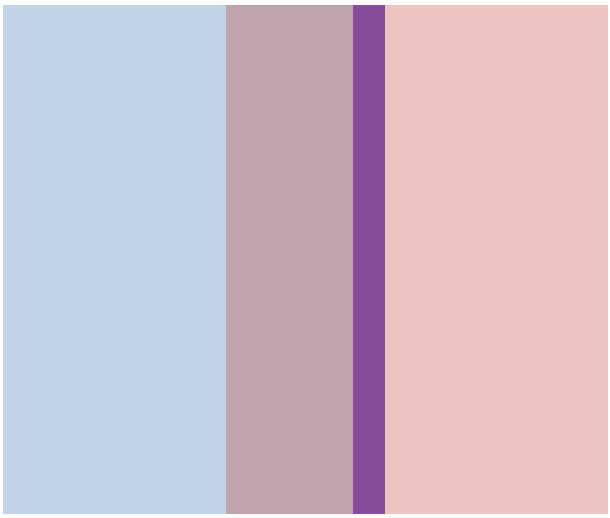
The last fix: change WHEN a camera is allowed to feel "fully confident"

OLD: fades only near the very edge



wide 50/50 zone
= visible double image

NEW: fades based on distance to EACH CAMERA'S OWN center



narrow crossover line
= one clean version wins

Overlap zones are HUGE (measured: 20-52% of the picture) — so with the old method, both cameras stayed "fully confident" across most of that shared area, blending two different (parallax-shifted) views of the same object.

Doesn't fix the underlying height-parallax — but now you see ONE clean (if imperfect) version instead of two overlapping ghosts.

SUMMARY

The Full Pipeline, Start to Finish